Techniki i narzędzia przetwarzania danych wielkoskalowych

Emilia Lubecka

Optymalizacje ...

- Wstęp – style architektury danych, w tym dane wielkoskalowe (big data)
- Analiza skupień (cluster analysis)
- Narzędzia - Apache Hadoop and Spark
- Python - NumPy, SciPy, Pandas, API PySpark
- Python - HPC
- Uczenie maszynowe
- **Optymalizacje kodu**

Jak przyspieszyć działanie programu?

- code optimization
- SIMD
- OpenMP

openMP

Parallelizing highly regular loops in matrix-oriented numerical programming.

First - 1997

The latest OpenMP 6.0 release was made in 2024 November

https://www.openmp.org/

openMP

Open Multi-processing (**OpenMP**)

is a technique of parallelizing a section(s) of C/C++/Fortran code.

OpenMP is also seen as an extension to C/C++/Fortran languages by adding the parallelizing features to them.

In general, OpenMP uses a portable, scalable model that gives programmers a simple and flexible interface for developing parallel applications for platforms that ranges from the normal desktop computer to the high-end supercomputers.

openMP

A process is created by the OS to execute a program with given resources(memory, registers);

generally, different processes do not share their memory with another.

A thread is a subset of a process, and it shares the resources of its parent process but has its own stack to keep track of function calls.

Multiple threads of a process will have access to the same memory.

openMP

OpenMP is the standard for programming on **shared-memory** architectures and comprises a set of compiler directives and libraries that control the parallel execution of the program.

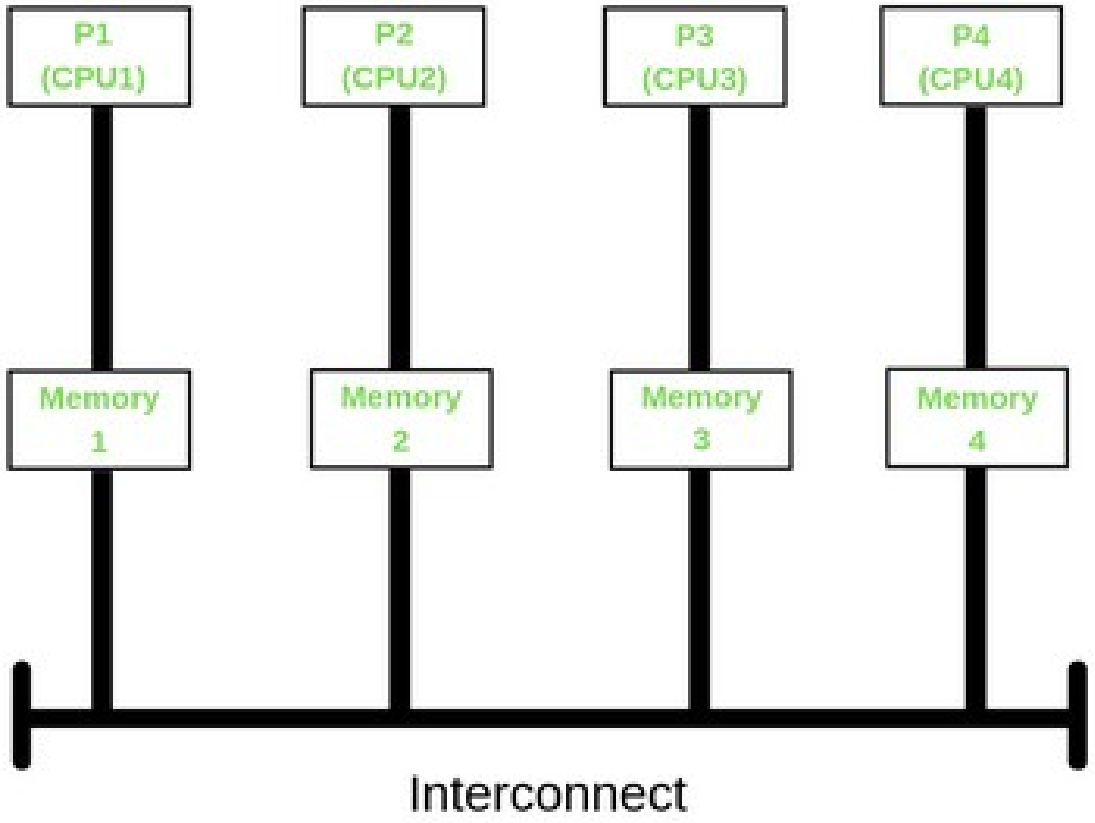
The advantage of implementing this standard is the **ease of conversion** of a serial program to a parallel program.

This is due to the fact that the programmer needs only to include OpenMP directives between certain code-blocks, as for or do loops.

Another advantage is that a program with OpenMP directives can be executed by both serial and parallel computers; because, the directives appear as comments to a compiler that does not conform to the OpenMP standard.

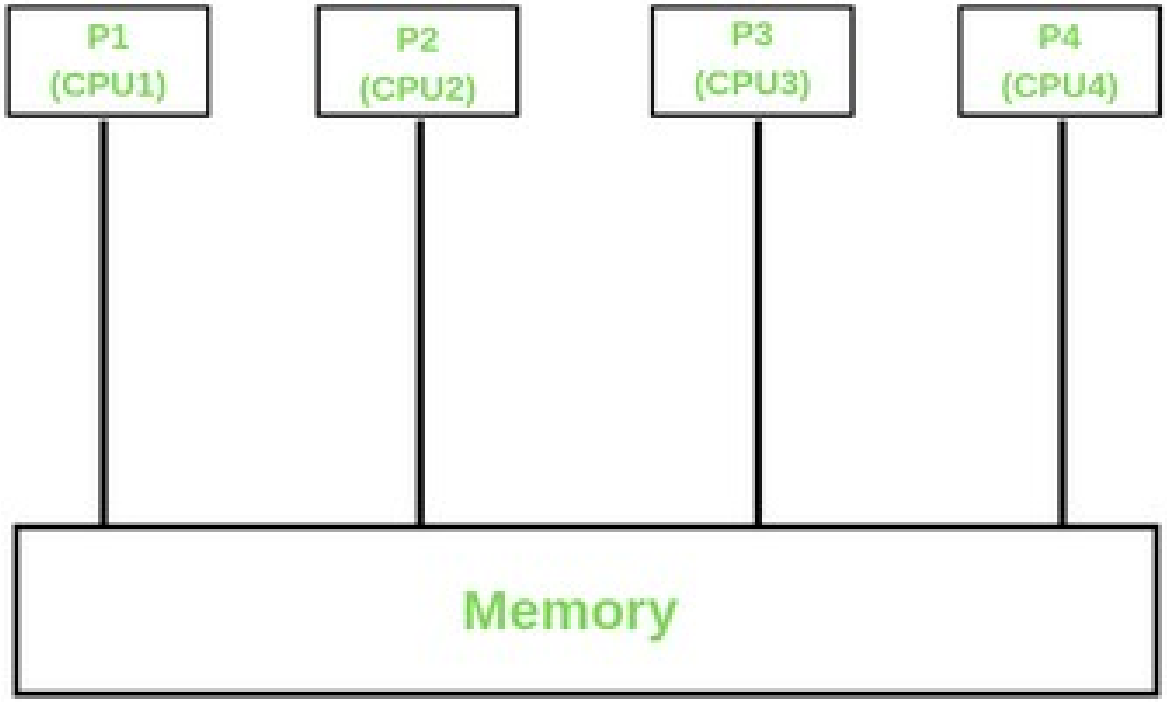
Thus, programs written with OpenMP directives are **portable**.

Distributed memory



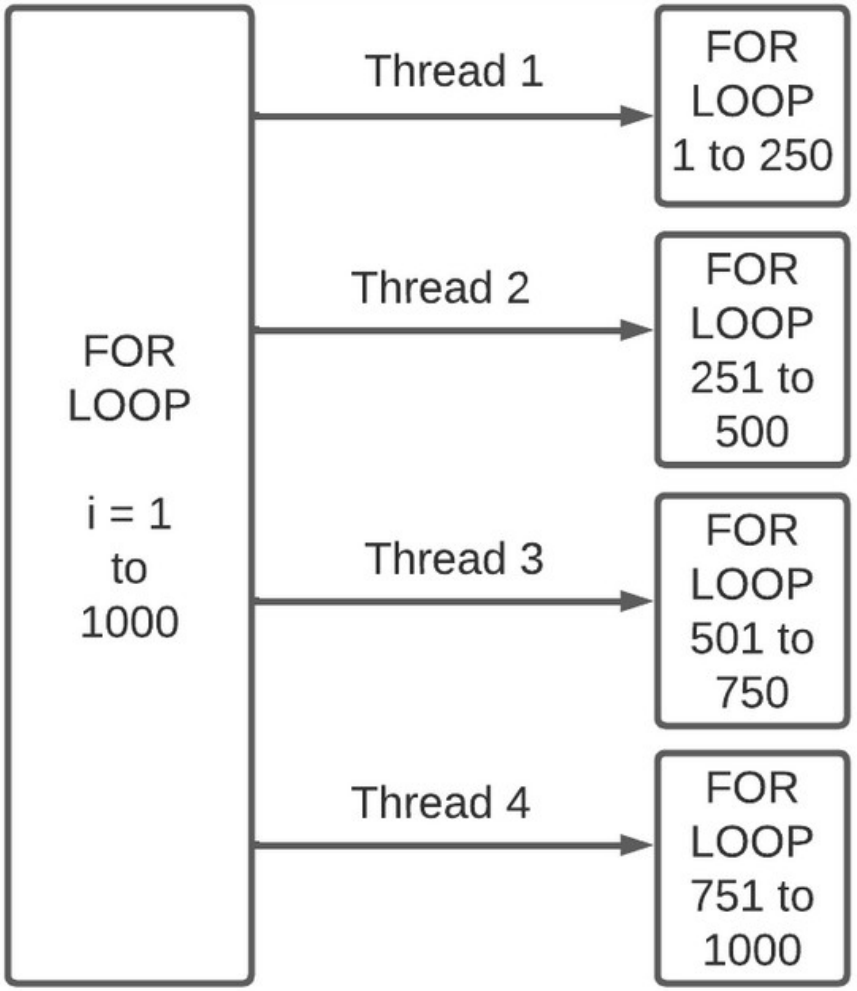
https://www.geeksforgeeks.org/openmp-introduction-with-installation-guide/

Shared memory



https://www.geeksforgeeks.org/openmp-introduction-with-installation-guide/

openMP - loop



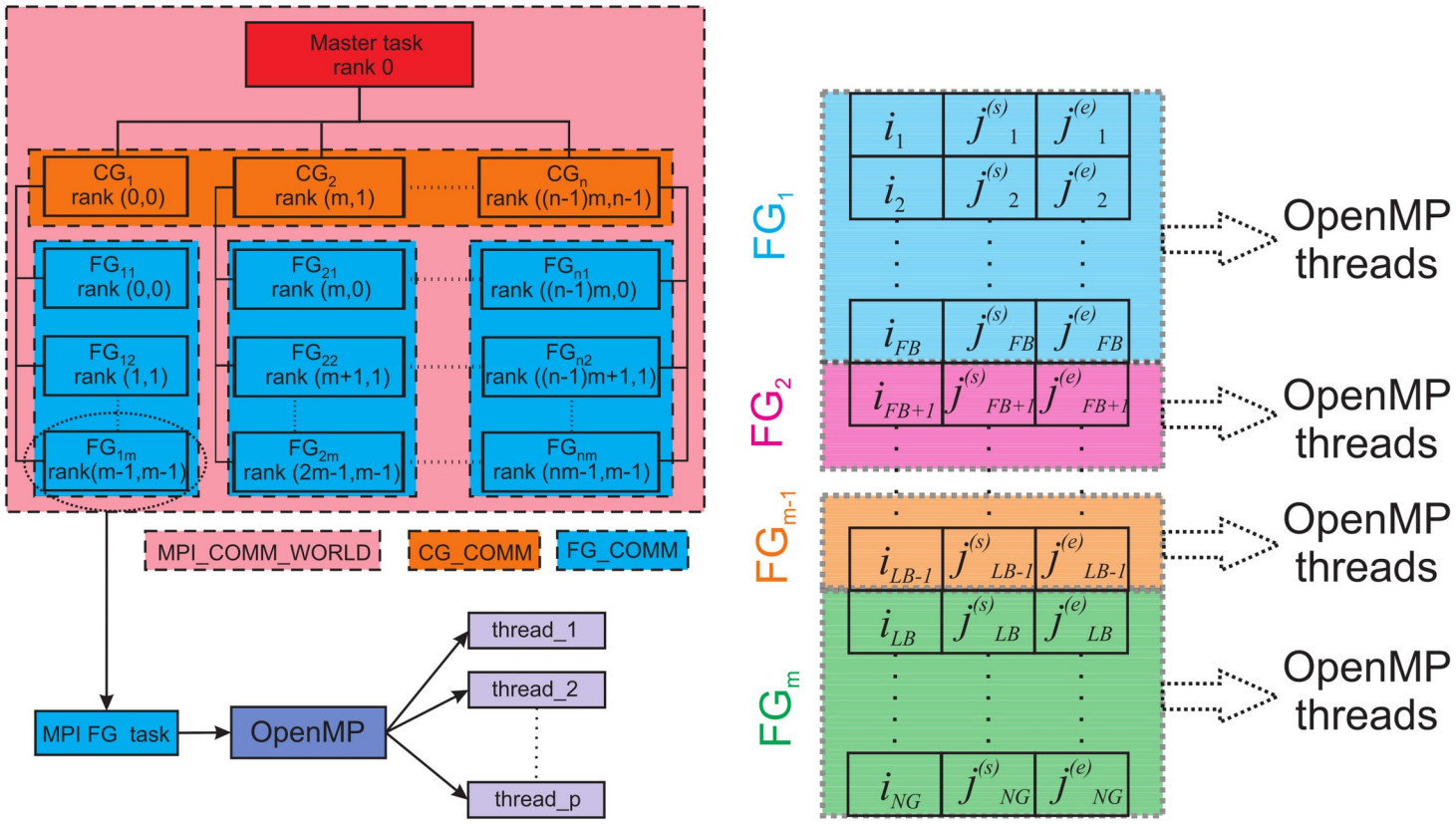
https://www.geeksforgeeks.org/c-parallel-for-loop-in-openmp/?ref=rp

openMP

```c
#include <omp.h>
#include <stdio.h>

int main()
{

    #pragma omp parallel for num_threads(4)
    for (int i = 1; i <= 10; i++) {
        int tid = omp_get_thread_num();
        printf("The thread %d  executes i = %d\n", tid, i);
    }

    return 0;
}
```

```
gcc -o aux.exe -fopenmp openMP.c
```

MPI + openMP



Sieradzan AK, et al. (2023) Optimization of parallel implementation of UNRES package for coarse-grained simulations to treat large proteins. *J. Comput. Chem.* 44(4): 602-625.

SIMD

Single (or Symmetric) instruction, multiple data (**SIMD**)

is a type of parallel processing.

SIMD can be internal (part of the hardware design) and it can be directly accessible through an instruction set architecture (ISA), but it should not be confused with an ISA.

SIMD describes computers with multiple processing elements that perform the same operation on multiple data points simultaneously.

Classification of parallel computers

Klasyfikacja komputerów równoległych (M. Flynn):

Podstawa liczba strumieni danych i strumieni rozkazów

- **SISD** - Single Instruction stream – Single Data stream

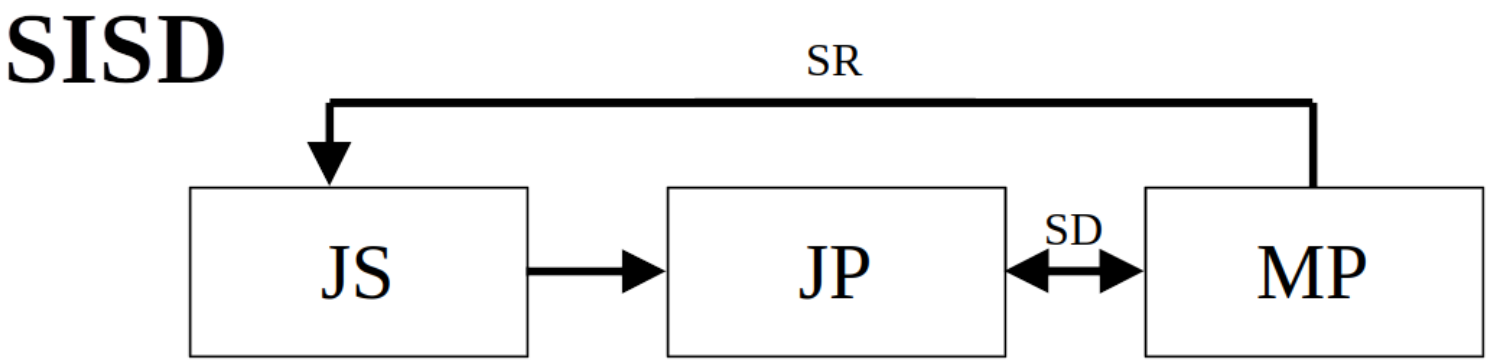
  Pojedynczy strumień rozkazów – pojedynczy strumień danych
- **SIMD** - Single Instruction stream – Multiple Data stream

  Pojedynczy strumień rozkazów – wielokrotny strumień danych
- **MISD** - Multiple instruction stream – Single Data stream

  Wielokrotny strumień rozkazów – pojedynczy strumień danych
- **MIMD** - Multiple instruction stream – Multiple Data stream

  Wielokrotny strumień rozkazów – wielokrotny strumień danych

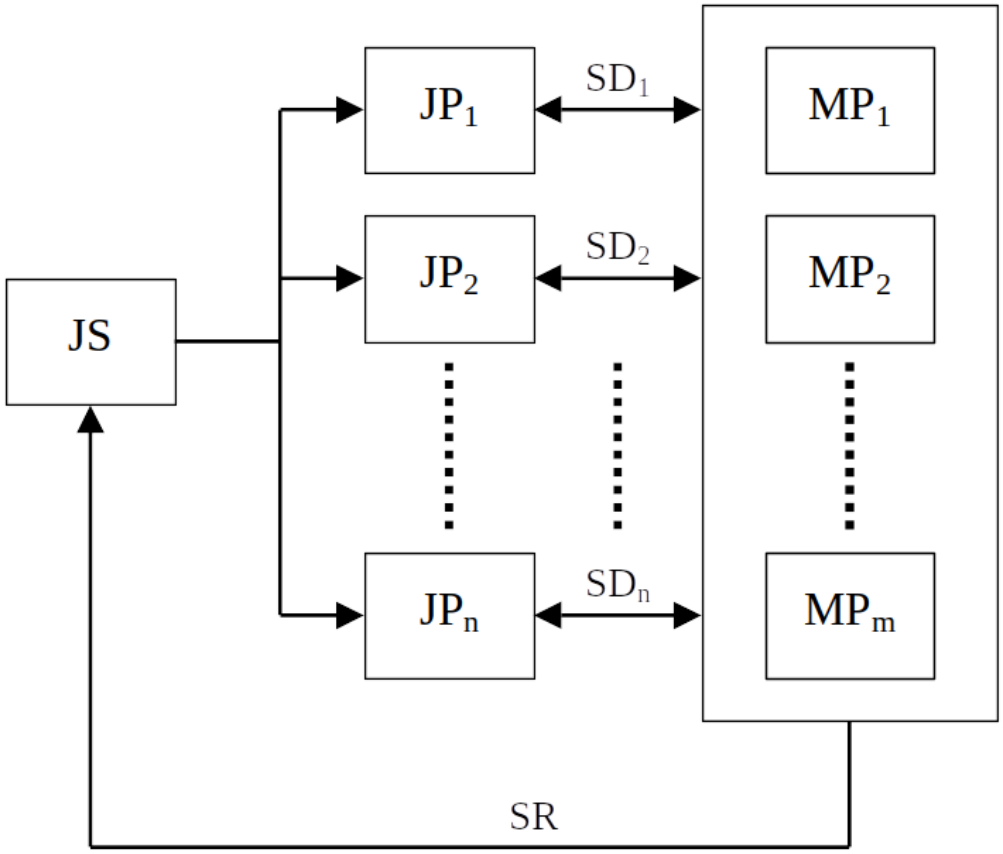
Single Instruction stream – Single Data stream



MP – moduł pamięci
JS – jednostka sterująca
JP – jednostka przetwarzająca
SR – strumienie rozkazów
SD – strumienie danych

Single Instruction stream – Multiple Data stream

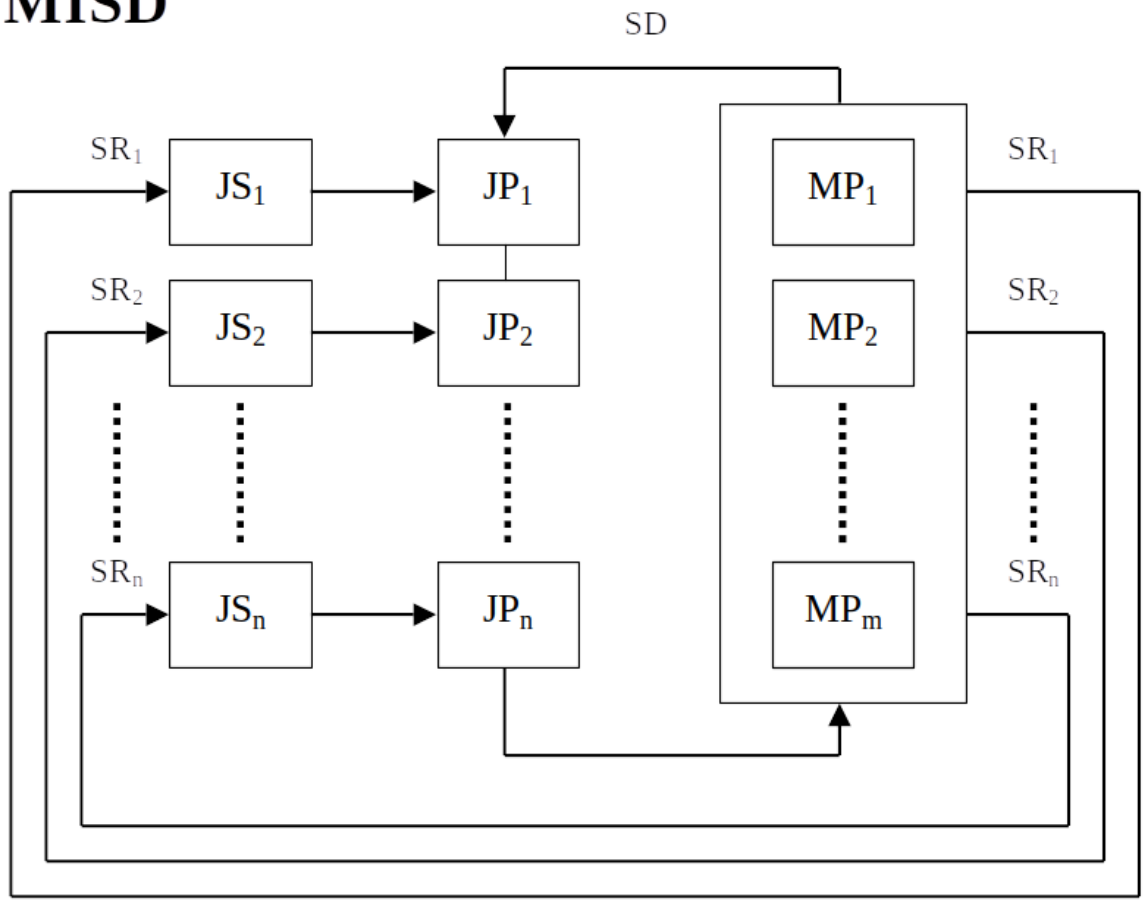
SIMD



MP – moduł pamięci
JS – jednostka sterująca
JP – jednostka przetwarzająca
SR – strumienie rozkazów
SD – strumienie danych

Multiple instruction stream – Single Data stream

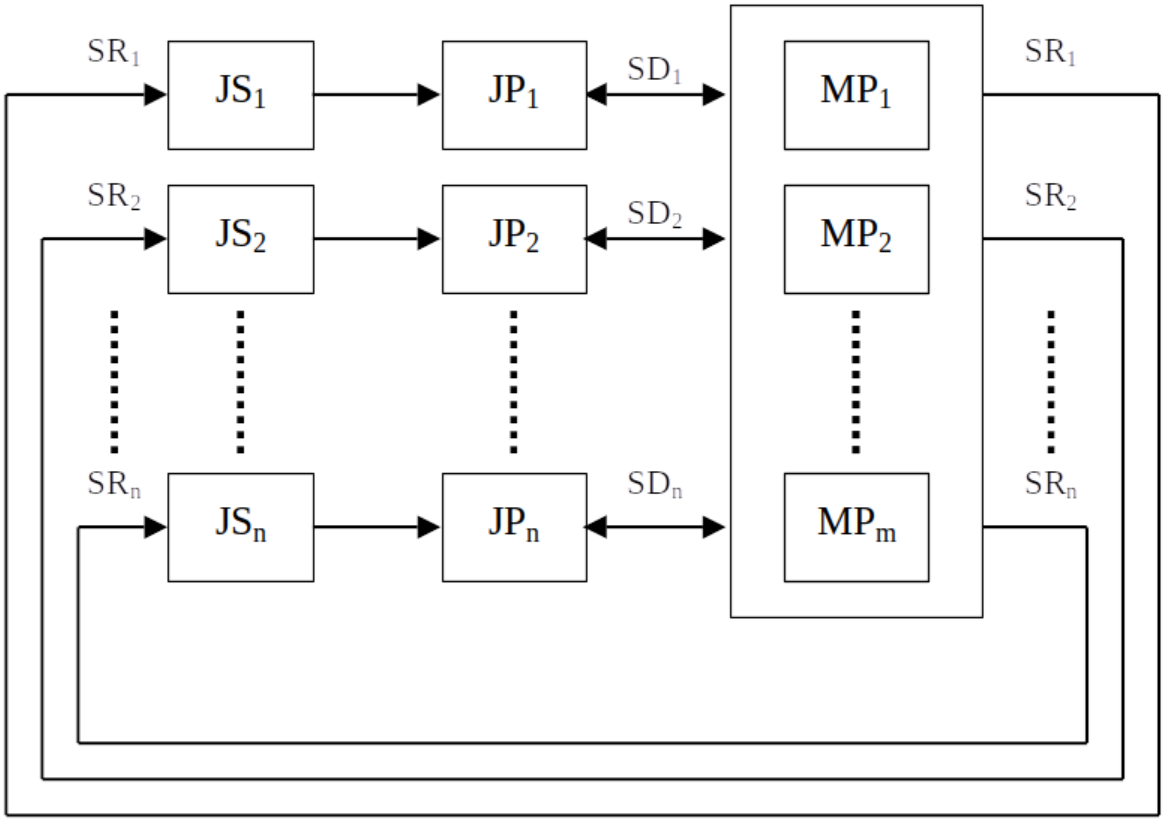
MISD



MP – moduł pamięci
JS – jednostka sterująca
JP – jednostka przetwarzająca
SR – strumienie rozkazów
SD – strumienie danych

Multiple instruction stream – Multiple Data stream

MIMD



MP – moduł pamięci
JS – jednostka sterująca
JP – jednostka przetwarzająca
SR – strumienie rozkazów
SD – strumienie danych

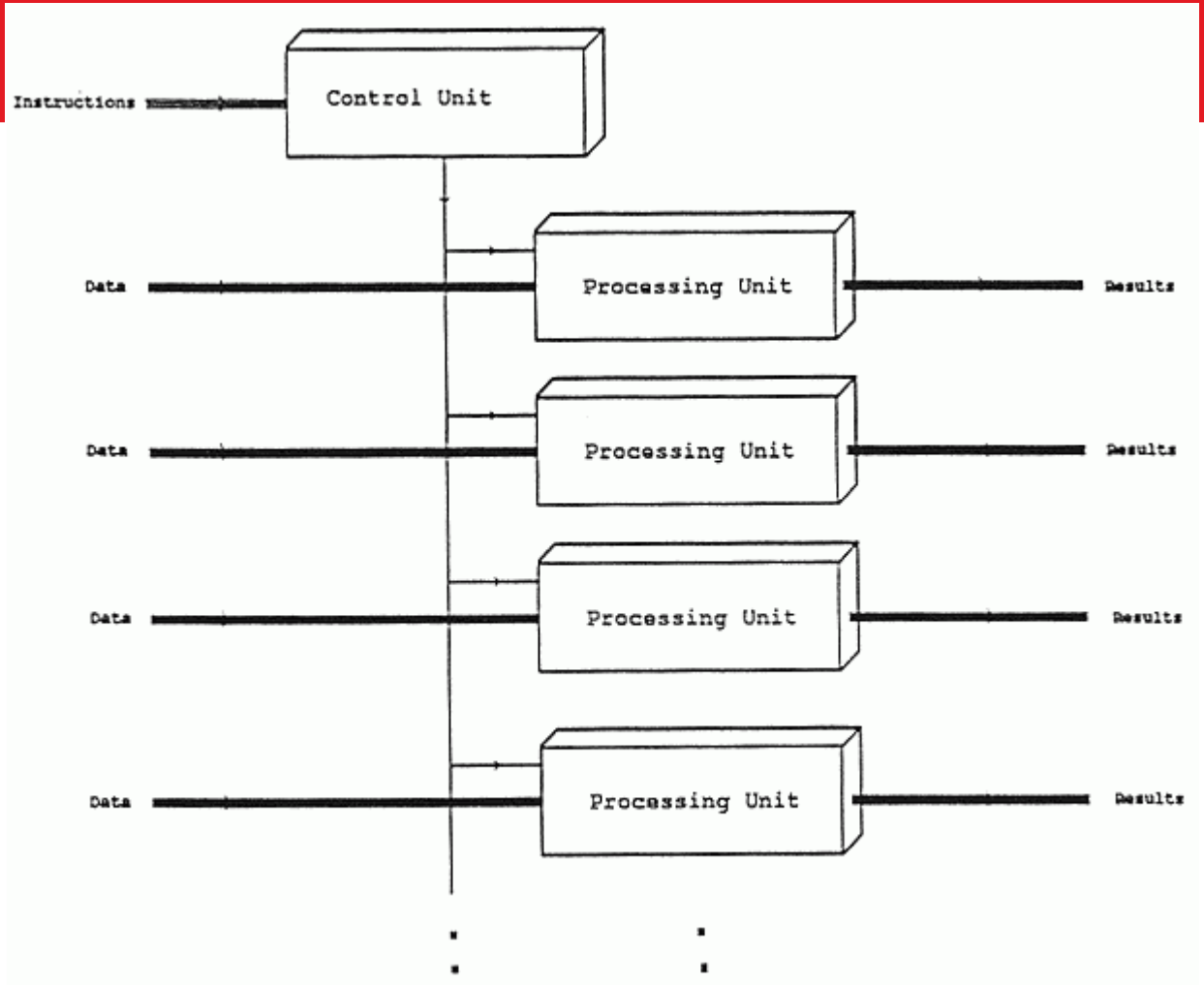
SIMD

The same set of instructions is executed in parallel to different sets of data.

This reduces the amount of hardware control logic needed by N times for the same amount of calculations, where N is the width of the SIMD unit.

http://ftp.cvut.cz/kernel/people/geoff/cell/ps3-linux-docs/CellProgrammingTutorial/BasicsOfSIMDProgramming.html

SIMD


Instructions
Control Unit
Data
Processing Unit
Results
Data
Processing Unit
Results
Data
Processing Unit
Results
Data
Processing Unit
Results

SIMD

Useful when the same value is being added to (or subtracted from) a large number of data points

With a SIMD processor there are two improvements to this process.

For one the data is understood to be in blocks, and a number of values can be loaded all at once. Instead of a series of instructions saying "retrieve this pixel, now retrieve the next pixel", a SIMD processor will have a single instruction that effectively says "retrieve n pixels" (where n is a number that varies from design to design).

SIMD

Disadvantages:

- Not all algorithms can be vectorized easily.

- Large register files which increases power consumption and required chip area.

- Currently, implementing an algorithm with SIMD instructions usually requires human labor; most compilers don't generate SIMD instructions from a typical C program, for instance.

SIMD in .NET

The .NET SIMD-accelerated types include the following types:

- The Vector2, Vector3, and Vector4 types, which represent vectors with 2, 3, and 4 Single values.

- Two matrix types, Matrix3x2, which represents a 3x2 matrix, and Matrix4x4, which represents a 4x4 matrix of Single values.

- The Plane type, which represents a plane in three-dimensional space using Single values.

SIMD in .NET

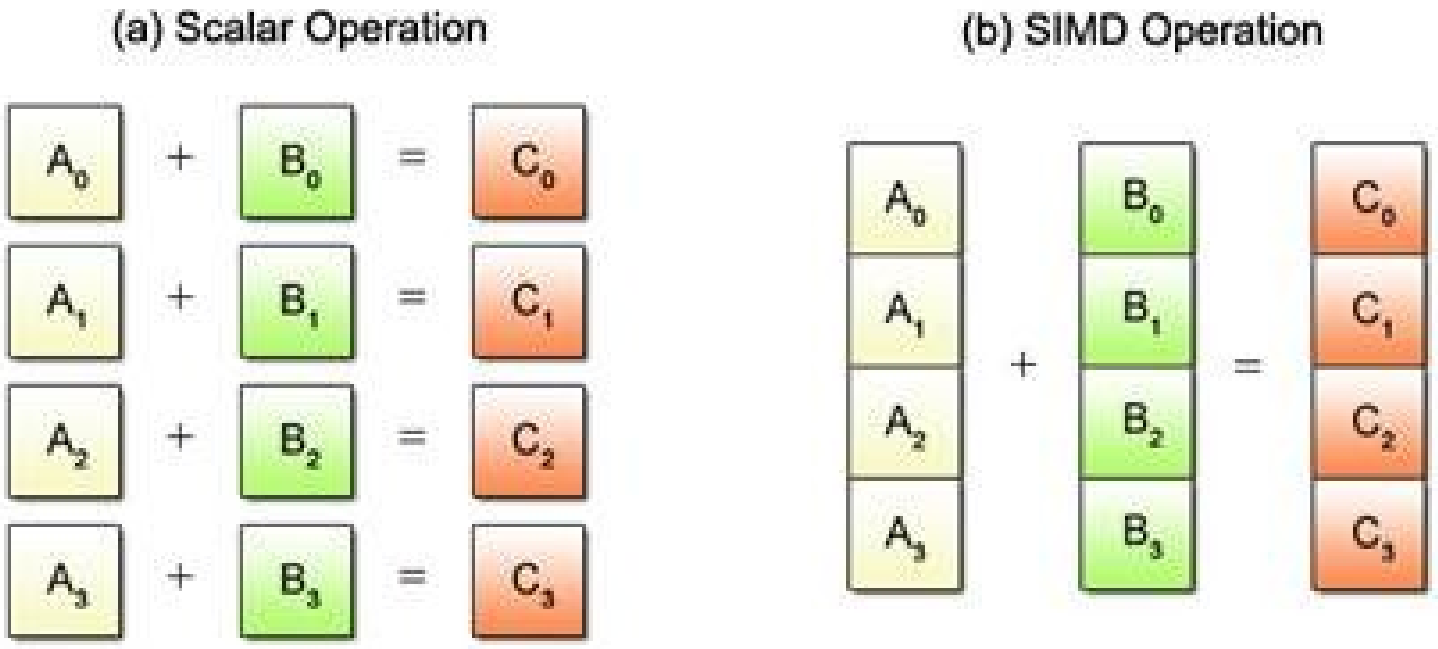
The .NET SIMD-accelerated types include the following types:

- The Quaternion type, which represents a vector that is used to encode three-dimensional physical rotations using Single values.

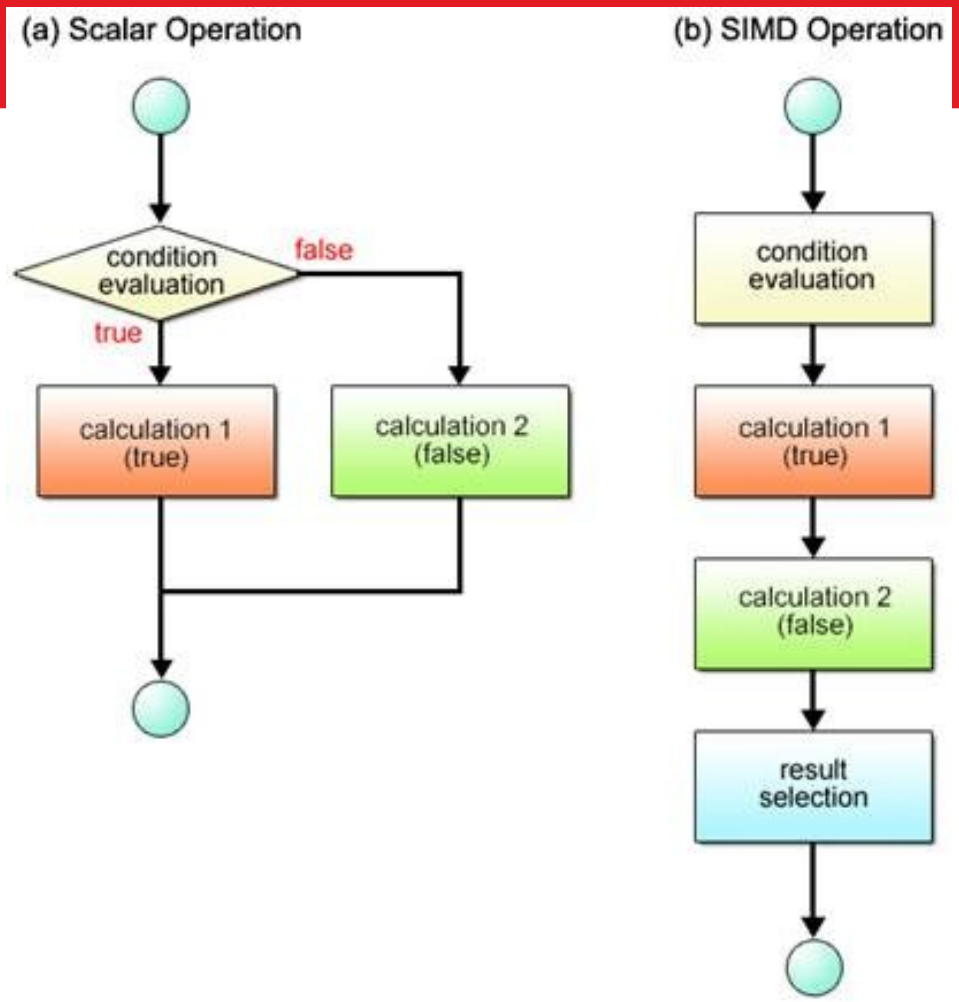
- The Vector<T> type, which represents a vector of a specified numeric type and provides a broad set of operators that benefit from SIMD support.

  The count of a Vector<T> instance is fixed for the lifetime of an application, but its value Vector<T>.Count depends on the CPU of the machine running the code.

SIMD


(a) Scalar Operation
A0 + B0 = C0
A1 + B1 = C1
A2 + B2 = C2
A3 + B3 = C3
(b) SIMD Operation
A0
A1
A2
A3
+
B0
B1
B2
B3
=
C0
C1
C2
C3

SIMD


(a) Scalar Operation
condition evaluation
false
true
calculation 1 (true)
calculation 2 (false)
(b) SIMD Operation
condition evaluation
calculation 1 (true)
calculation 2 (false)
result selection

SIMD

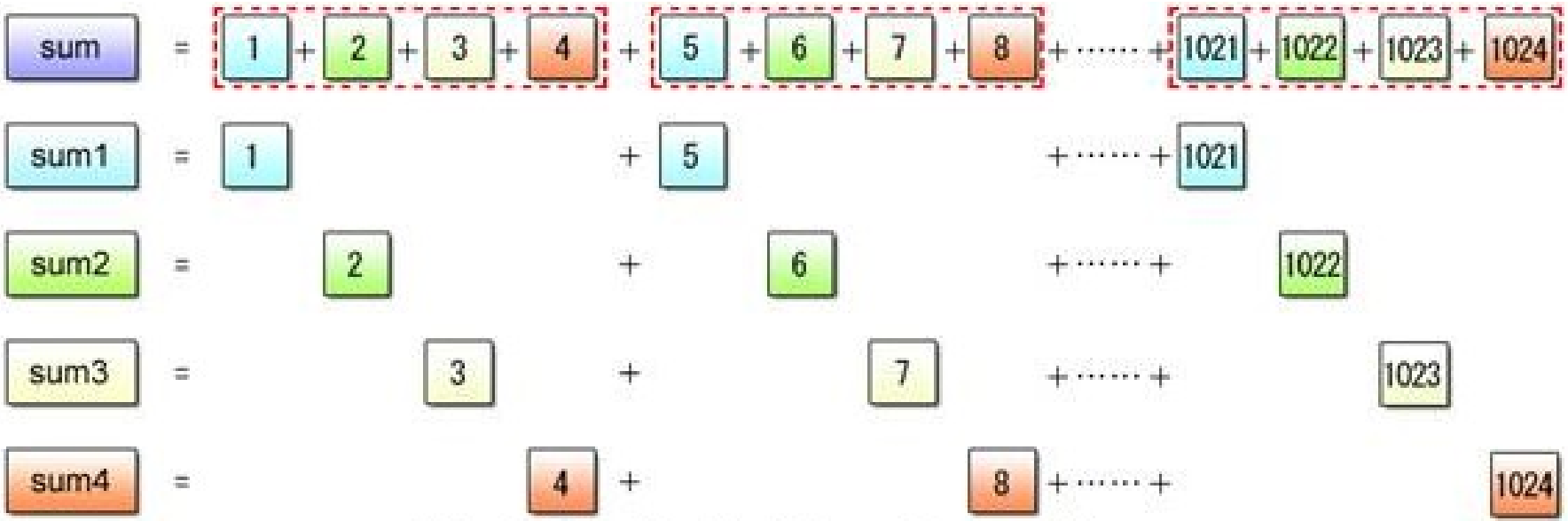


Fig. 2.18: Partial Sum Calculation

SIMD

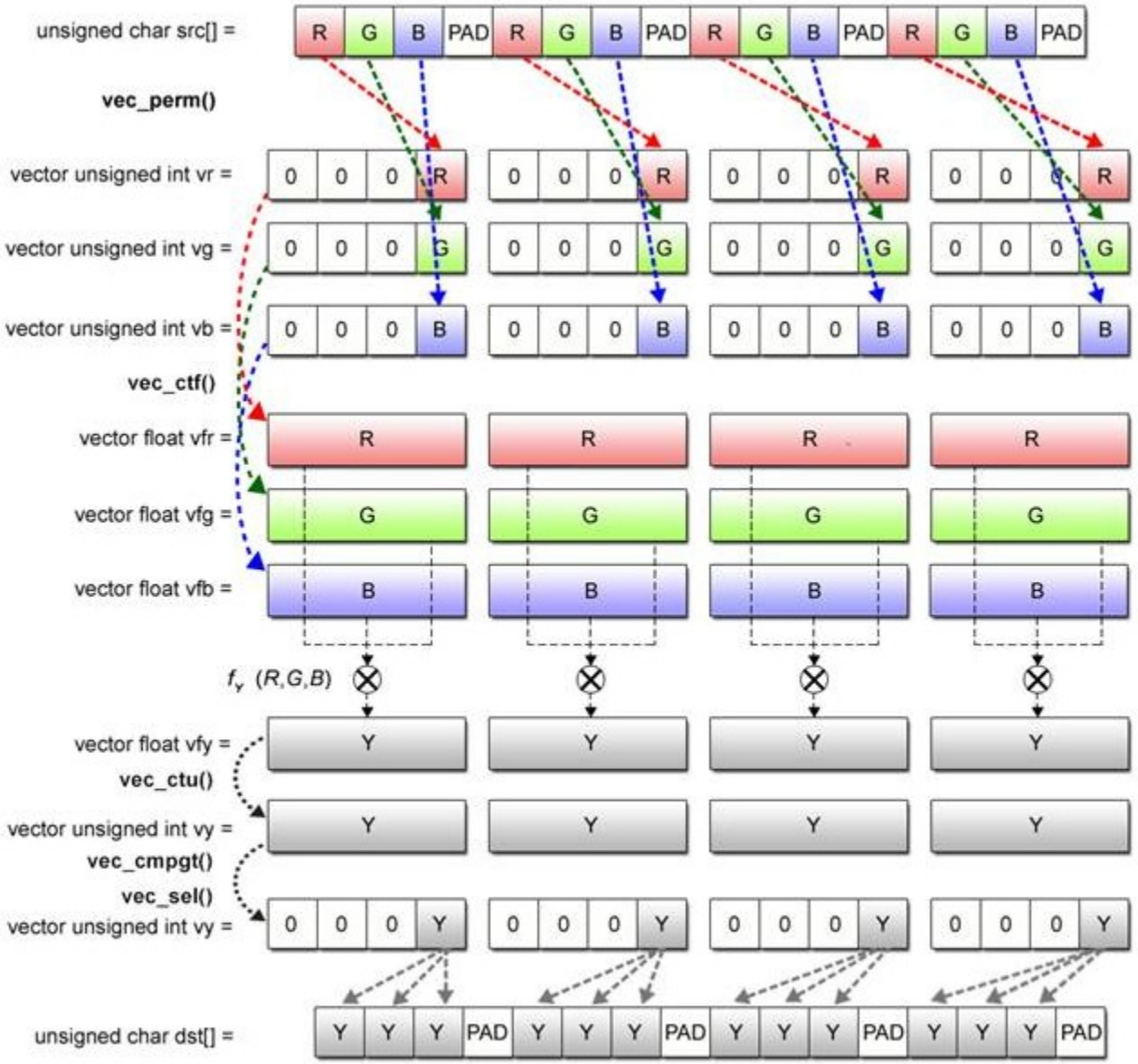


Fig. 2.27: Grayscale Conversion Processing by SIMD Operations

OpenMP with SIMD

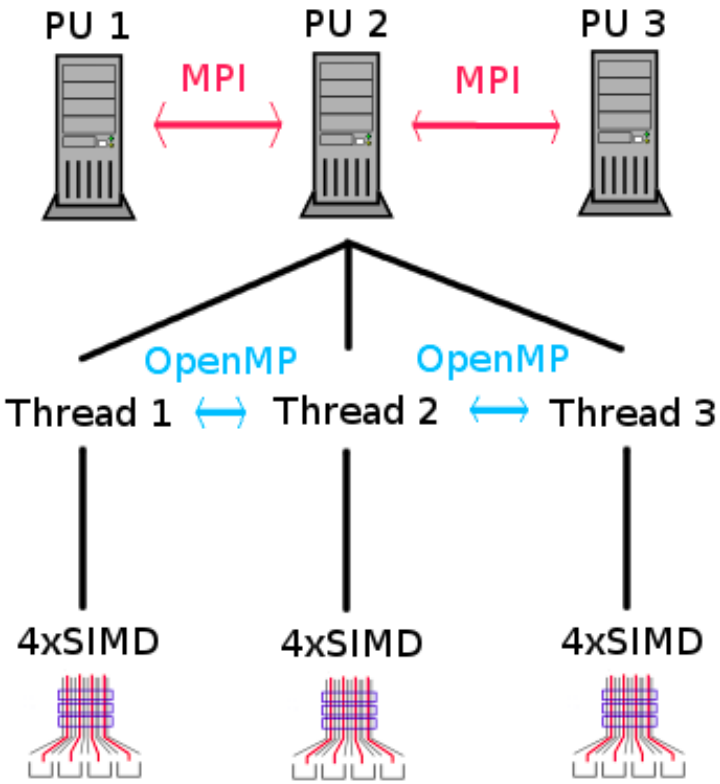
The simd construct can be applied to a loop to indicate that the loop can be transformed into a SIMD loop (that is, multiple iterations of the loop can be executed concurrently using SIMD instructions).

#pragma omp simd [clause[[,] clause] ...] new-line

for-loops

SIMD in openMP

A parallel program running on three computation units with three threads each with width 4 SIMD unit in each thread



Loops

You have a 2-dimensional array, but memory in the computer is inherently 1-dimensional. So while you imagine your array like this:

```
0,0 | 0,1 | 0,2 | 0,3
----+-----+-----+----
1,0 | 1,1 | 1,2 | 1,3
----+-----+-----+----
2,0 | 2,1 | 2,2 | 2,3
```

Your computer stores it (in C) in memory as a single line:

0,0 | 0,1 | 0,2 | 0,3 | 1,0 | 1,1 | 1,2 | 1,3 | 2,0 | 2,1 | 2,2 | 2,3

GDAŃSK UNIVERSITY
OF TECHNOLOGY

HISTORY IS WISDOM
FUTURE IS CHALLENGE